\newpage

1. What is civil3d-mcp?

civil3d-mcp is a Model Context Protocol (MCP) bridge that exposes the Autodesk Civil 3D 2025, 2026 and 2027 .NET APIs to Claude Desktop (or any MCP client). Once installed, you can drive Civil 3D in plain English:

> *"List the alignments in the active drawing." "Sample the existing-ground surface elevation at station 1+250 on alignment R1." "Export all COGO points from the BENCHMARKS group to `C:\Projects\survey.csv`."*

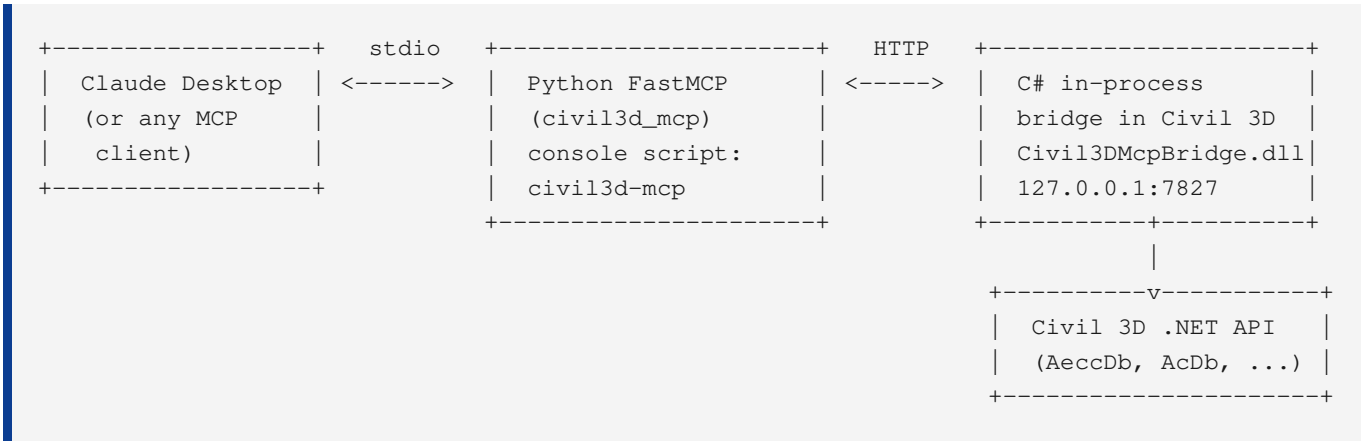
The bridge translates your natural-language requests into typed, schema-validated API calls. Civil 3D executes them on its main UI thread inside a transaction, and the response comes back to Claude as JSON.

Highlights

- **110 Civil 3D tools**, **128 in the MCP server** including design checks, reports and calculators. Domains include alignments, profiles, corridors, surfaces, sections and earthworks, points, pipe and pressure networks, blocks and signage, road markings, mass haul, drafting and drawing utilities.
- **Local-only** - the bridge listens on `127.0.0.1` only (ports 7825 for 2025, 7826 for 2026, 7827 for 2027). Nothing leaves your machine.
- **Civil 3D 2025, 2026 and 2027** in one bundle; the 2027 build targets the .NET 10 API.
- **Schema-validated** - every tool argument is a Pydantic model, so Claude knows exactly what each tool accepts.

\newpage

2. Architecture



Three layers:

1. **C# add-in** (`Civil3DMcpBridge.dll`) - a NetLoadable bundle that boots an HTTP listener the moment Civil 3D loads it. Every tool call is marshalled to the Civil 3D main UI thread inside a `LockDocument()` + `Transaction` scope.
2. **Python MCP server** (`civil3d-mcp` console script) - FastMCP wrapper. Each tool is a thin Pydantic-validated proxy that POSTs JSON to the bridge.
3. **Claude Desktop** (or any MCP client) - runs the Python server over stdio and presents the tools as natural-language-callable functions.

\newpage

3. System requirements

Component	Minimum
OS	Windows 10/11 x64
Civil 3D	2025, 2026 or 2027 (with the Civil 3D vertical, not just AutoCAD)
.NET SDK	.NET 8 (only needed at install time, to build the C# add-in)
Python	3.11+
Claude Desktop	Latest
Disk	< 50 MB
Network	None - `127.0.0.1` only

Important: You need a *real* Civil 3D install. The bridge depends on `AeccDbMgd.dll` from `C:\Program Files\Autodesk\AutoCAD 2026\C3D\`, which only ships with Civil 3D, not plain AutoCAD.

\newpage

4. Installation

4.1 One-shot installer (recommended)

From a PowerShell prompt in the cloned repo root:

```
powershell -ExecutionPolicy Bypass -File .\install.ps1
```

The installer will:

1. Verify prerequisites (Civil 3D path, .NET 8 SDK, Python 3.11+).

2. `dotnet build -c Release` the plugin.
3. Deploy the bundle to `%APPDATA%\Autodesk\ApplicationPlugins\Civil3DMcpBridge.bundle\Contents\`.
4. `pip install -e server\` - creates the `civil3d-mcp` console script.
5. Merge a `civil3d` entry into `%APPDATA%\Claude\claude_desktop_config.json`.

Available switches:

- `-Civil3DPath "C:\Program Files\Autodesk\AutoCAD 2025"` - point at a different Civil 3D install.
- `-PerMachine` - install the bundle under `C:\ProgramData` instead of `%APPDATA%` (needs admin).
- `-SkipBuild` - reuse a previously-built DLL.
- `-SkipPython` - plugin-only deployment (you'll wire up the Python server manually).
- `-PythonExe python3.11` - pin to a specific interpreter.

4.2 Manual install

If the script can't run for some reason:

```
# 1. Build the add-in
cd plugin
dotnet build -c Release

# 2. Deploy the bundle
$bundle = "$env:APPDATA\Autodesk\ApplicationPlugins\Civil3DMcpBridge.bundle"
$contents = "$bundle\Contents"
mkdir $contents -Force
copy PackageContents.xml "$bundle\PackageContents.xml"
copy bin\Release\net8.0-windows\Civil3DMcpBridge.* $contents

# 3. Install Python package
cd ..\server
pip install -e .

# 4. Wire up Claude Desktop config (see section 5 below)
```

\newpage

5. Configuring Claude Desktop

Open `%APPDATA%\Claude\claude_desktop_config.json` (full path: `C:\Users\<you>\AppData\Roaming\Claude\claude_desktop_config.json`). The file must contain **one** top-level JSON object. Add a `civil3d` entry under `mcpServers`:

```
{
  "mcpServers": {
    "civil3d": {
      "command": "C:\\Users\\<you>\\AppData\\Local\\Packages\\PythonSoftwareFoundation.Pytho
```

```
      "env": {
        "CIVIL3D_BRIDGE_URL": "http://127.0.0.1:7827"
      }
    }
  }
}
```

If you already have other MCP servers configured (e.g. filesystem, github, gmail), put the `"civil3d": {...}` entry as a sibling inside the same `mcpServers` object - separated by a comma - not in a second top-level object.

Common JSON pitfalls:

- **Backslashes must be doubled** (`\\`) inside JSON strings.
- **Two top-level objects** glued back-to-back (`{...}{...}`) is invalid
    - it triggers Claude's "Could not load app settings - Unexpected token" error. Wrap everything in one outer `{...}`.
- The path to `civil3d-mcp.exe` differs depending on whether you used the Microsoft Store Python (path above) or system Python (`C:\Python313\Scripts\civil3d-mcp.exe`). Find yours with:

  ```
  (Get-Command civil3d-mcp).Path
  ```

After saving, **fully quit** Claude Desktop (system tray → Quit) and reopen. The hammer icon in the bottom-right of the chat input should now show a `civil3d` server with 128 tools.

\newpage

6. Verifying the install

6.1 Check the bridge is loaded in Civil 3D

1. Start Civil 3D and open any DWG.
2. At the C3D command line, type `MCPSTATUS` and press Enter.
3. You should see: `civil3d-mcp bridge: listening on http://127.0.0.1:7827`

If you don't see that:

- Check `%APPDATA%\Autodesk\ApplicationPlugins\Civil3DMcpBridge.bundle\` exists and contains `Contents\Civil3DMcpBridge.dll`.
- Look in C3D's command-line history for a load error after startup.
- Try `NETLOAD` and pick the DLL manually to surface the error.

6.2 Hit the health endpoint

In a *separate* PowerShell window (Civil 3D must be running):

```
curl http://127.0.0.1:7827/health
```

Expected:

```
{
  "ok": true,
  "version": "0.4.0",
  "toolCount": 110,
  "tools": ["civil3d_add_baseline_region", "...", "civil3d_zoom_to_alignment"]
}
```

6.3 First conversation with Claude

In Claude Desktop:

> **You:** *Run the civil3d bridge health check.*
>
> **Claude:** Calls `civil3d_bridge_health` -> returns `{"ok": true, "version": "0.4.0", "toolCount": 110}`. "The bridge is healthy and exposing 110 Civil 3D tools."

Then try a real query:

> **You:** *List the layers in the active drawing.*
>
> **Claude:** Calls `civil3d_list_layers` -> returns a list of all layers with their state.

\newpage

7. Tool reference

The MCP server exposes **128 tools**. **110 run inside Civil 3D** through the add-in: the tools that read or change the drawing. The other **18** run in the Python server: instance and document selection, four reports and design checks that use bridge data, and seven COGO, curve and earthwork calculators that need no Civil 3D at all.

`R` = read-only (queries the drawing). `W` = write (changes the drawing inside a transaction; Civil 3D's UNDO reverses it).

7.1 Sessions, instances and undo (8)

Tool	R/W	Purpose
`civil3d_activate_document`	W	Bring an open drawing to the front (makes it the active MDI document)
`civil3d_bridge_health`	R	Check the bridge for the selected Civil 3D session: bridge version, Civil 3D release, .NET runtime, open drawings, busy state and the registered tool list
`civil3d_call`	W	Call any tool registered with the C# bridge by name with raw args
`civil3d_list_documents`	R	Open drawings in the selected Civil 3D session, the active one, and any command in progress
`civil3d_list_instances`	R	List every running Civil 3D session with the bridge loaded (release, pid, port), and which one calls currently go to
`civil3d_select_document`	R	Route following calls to a specific open drawing (file name or full path) without switching windows
`civil3d_select_instance`	R	Pin all following calls to one Civil 3D session, by release (e.g. 2027), process id, port or URL. Pass clear=true to return to automatic selection
`civil3d_undo`	W	Queue UNDO for N steps in the target drawing (each MCP write is normally one step)

7.2 Alignments and horizontal design (14)

Tool	R/W	Purpose
`civil3d_add_station_equation`	W	Add a station equation (e.g. to tie to an existing chainage at a section break)
`civil3d_create_alignment_by_pis`	W	Create a horizontal alignment from PI coordinates with a radius per interior PI and optional symmetric clothoid spirals (spiral-curve-spiral)
`civil3d_create_alignment_from_polyline`	W	Convert an existing AutoCAD polyline (identified by its handle) into a Civil 3D alignment
`civil3d_create_offset_alignment`	W	Create a constant-offset alignment from a parent (e.g. lane edge from CL)
`civil3d_export_alignment_geometry`	R	Dump alignment entity table (line/curve/spiral) to CSV for external review
`civil3d_get_alignment_geometry_detail`	R	Every tangent, arc and spiral with stations, lengths, radii, deflections, spiral A-values, coordinates, plus station equations
`civil3d_get_alignment_info`	R	Geometry entities, station equations, design speeds, child profile names
`civil3d_get_design_speeds`	R	Design speed table assigned to an alignment
`civil3d_get_station_at_xy`	R	Reverse lookup - given a world (X, Y), return the station + perpendicular offset on the given alignment
`civil3d_get_superelevation`	R	Superelevation curves and critical stations with lane cross-slopes (%)

`civil3d_get_xy_at_station`	R	Just the (easting, northing) for (alignment, station, offset)
`civil3d_list_alignments`	R	List every alignment in the active drawing with stationing and style
`civil3d_point_at_chainage`	R	Return (easting, northing, optional elevation, tangent direction) at a given alignment station and lateral offset
`civil3d_set_design_speed`	W	Add or replace a design speed entry on an alignment

7.3 Profiles and vertical design (13)

Tool	R/W	Purpose
`civil3d_add_pvi`	W	Add a PVI to a layout profile at (station, elevation)
`civil3d_add_vertical_curve`	W	Insert a fixed-length parabolic vertical curve at a PVI. Sag/crest inferred from surrounding grades; rolls back if expected_curve_type is supplied and the inferred type doesn't match
`civil3d_create_layout_profile`	W	Create an empty layout (design / FG) profile
`civil3d_create_profile_by_pvis`	W	Build a layout (design) profile from [station, elevation] PVIs and a symmetric parabola length per interior PVI
`civil3d_create_profile_from_surface`	W	Standard EG profile creation - sample a surface along an alignment
`civil3d_create_profile_view`	W	Create a profile view at a chosen drawing insertion point
`civil3d_delete_pvi`	W	Remove the PVI nearest a station (within tolerance) from a layout profile
`civil3d_get_elevation_at_station`	R	Sample profile elevations at one or more stations (batched)
`civil3d_get_profile_geometry_detail`	R	Tangents (grades) and vertical curves (type, length, K, A) plus the PVI table
`civil3d_get_profile_info`	R	Get the entity list and PVI table for one profile
`civil3d_list_profiles`	R	List the profiles on a given alignment (EG, FG, layout, etc.)
`civil3d_sample_profile`	R	Elevation and grade at a fixed interval along a profile; optional CSV for long-section tables
`civil3d_sample_surface_along_alignment`	R	Walk a surface along an alignment at fixed interval, returning elevations at the supplied lateral offsets

7.4 Corridors and assemblies (12)

Tool	R/W	Purpose
`civil3d_add_baseline_region`	W	Append a region (with assembly) to an existing baseline

civil3d_create_corridor	W	Create a corridor from (alignment, profile, assembly)
civil3d_create_corridor_surface	W	Create a corridor surface (e.g
civil3d_export_corridor_sections	W	Sample cross-sections at fixed station interval and dump every corridor point code to CSV (station, point_code, offset, elevation, easting, northing)
civil3d_get_corridor_codes	R	Link, point and shape codes present in a corridor (needed to build surfaces)
civil3d_get_corridor_info	R	Full baseline/region/assembly map of a corridor
civil3d_get_corridor_regions	R	Baselines (alignment/profile) and regions with station ranges and assemblies
civil3d_list_corridor_surfaces	R	Corridor surfaces with their link / feature line codes and boundaries
civil3d_list_corridors	R	List corridors with baseline count and out-of-date flag
civil3d_rebuild_corridor	W	Rebuild a corridor's model
civil3d_set_region_assembly	W	Apply a different assembly to a region - standard urban→rural transition
civil3d_set_region_target_surface	W	Bind a named subassembly target (typically 'Daylight_Target') on a region to an existing surface

7.5 Surfaces and feature lines (15)

Tool	R/W	Purpose
civil3d_add_breakline_from_polyline	W	Add an AutoCAD polyline as a breakline on a TIN surface
civil3d_add_surface_boundary	W	Add an outer / hide / show / data-clip boundary from a closed polyline and rebuild
civil3d_analyze_surface_slopes	R	Area-weighted slope bands (%) and optional elevation bands from the TIN triangles
civil3d_create_feature_line	W	Create a feature line from an existing polyline (handle) or from [x, y, z] points
civil3d_create_persistent_volume_surface	W	Create a permanent TIN volume surface between two existing surfaces (lives in the drawing for ongoing cut/fill tracking)
civil3d_create_tin_surface	W	Create an empty TIN surface
civil3d_extract_surface_contours	W	Extract contour polylines at an interval, optionally onto a layer
civil3d_feature_line_elevations_from_surface	W	Assign feature line elevations from a surface, optionally inserting grade breaks
civil3d_get_elevation_at_xy	R	Sample a surface elevation at a world (X, Y)
civil3d_get_surface_statistics	R	Point count, min/max/mean elevation, extents, 2D/3D area and grade statistics

`civil3d_get_surface_volume`	W	Compute cut/fill/net volume between two TIN surfaces
`civil3d_list_feature_lines`	R	Feature lines in all sites and site-less, with 2D/3D length and elevation range
`civil3d_list_surfaces`	R	List TIN/grid surfaces with elevation range and 2D/3D area
`civil3d_rebuild_surface`	W	Rebuild an out-of-date surface
`civil3d_sample_surface_transect`	R	Elevations along a line (start/end/step) or at a list of points

7.6 Sections, earthworks and quantities (8)

Tool	R/W	Purpose
`civil3d_compute_quantity_takeoff`	R	Read per-section cut/fill/net from a computed material list on a sample line group, plus cumulative totals
`civil3d_create_sample_line_group`	W	Create a sample line group on an alignment, populated by station range at a fixed interval
`civil3d_create_section_views`	W	Create one section view per sample line, laid out in a grid from an origin
`civil3d_earthwork_by_sections`	W	Cut/fill volumes between an existing and a design surface along an alignment by average end area, sampled straight from the surfaces (no sample lines or QTO setup)
`civil3d_export_mass_haul_csv`	W	Dump the mass-haul table (station, cut, fill, net, cumulative) to CSV
`civil3d_export_sampled_sections`	W	Offset/elevation points of every sampled section (EG, FG, corridor surfaces) per station; write a CSV for cross-section QA or third-party volume checks
`civil3d_list_sample_line_groups`	R	List sample line groups, optionally filtered to one alignment
`civil3d_list_sample_lines`	R	Sample lines in a group with station, swath offsets and section count

7.7 Points and point groups (9)

Tool	R/W	Purpose
`civil3d_add_points_to_surface`	W	Attach a point group (or all CogoPoints) as a data source on an existing TIN surface, then rebuild it
`civil3d_create_point`	W	Create one COGO point at (northing, easting, elevation)
`civil3d_create_point_group`	W	Create a point group, optionally filtered by raw-description glob (e.g. 'IP-*' for intersection points only)
`civil3d_create_surface_from_points`	W	One-shot: create a TIN surface from a named point group (or all CogoPoints if no group specified) and rebuild

civil3d_export_points_csv	W	Export all COGO points to CSV in PNEZD format
civil3d_get_feature_line_points	W	All PI and elevation points with running 2D chainage; optional CSV
civil3d_import_points_csv	W	Bulk-create COGO points from CSV (PNEZD/PENZD with header detection)
civil3d_list_point_groups	R	List point groups in the drawing
civil3d_list_points	R	Paginated list of COGO points

7.8 Pipe and pressure networks (7)

Tool	R/W	Purpose
civil3d_create_culvert_at_chainage	W	Place a complete culvert across an alignment from chainage + sizing
civil3d_create_junction_corner_fillets	W	Draw a right-angle junction layout at a crossing point: four filleted kerb corners (tangent arcs) of the given radius, plus optional approach kerb lines and a central channelizing island
civil3d_get_pipe_network_info	R	Full structure and pipe table for a named network
civil3d_get_pressure_network_schedule	W	Pipe lengths grouped by part and nominal diameter; optional CSV
civil3d_list_pipe_networks	R	List pipe networks with pipe + structure counts
civil3d_list_pipe_parts	R	List pipe + structure families and sizes available in the drawing's parts list
civil3d_list_pressure_networks	R	Pressure (water) networks with pipe, fitting and appurtenance counts

7.9 Blocks, signage and road markings (7)

Tool	R/W	Purpose
civil3d_create_km_post_series	W	Place KM (or hectometre) posts at fixed interval along an alignment
civil3d_create_signpost_at_chainage	W	Opinionated wrapper for road sign placement
civil3d_draw_lane_marking	W	Draw a longitudinal lane marking polyline parallel to an alignment
civil3d_draw_pedestrian_crossing	W	Draw a zebra crossing as perpendicular stripes centred on a station
civil3d_insert_block_at_chainage	W	Insert a block reference at (alignment, station, offset) with rotation matching alignment direction (along/perpendicular/absolute)
civil3d_insert_blocks_batch	W	Bulk insert blocks

civil3d_list_blocks	R	List block definitions in the drawing, optionally filtered by name substring

7.10 Roundabouts and vehicle tracking (3)

Tool	R/W	Purpose
civil3d_create_roundabout_centerline	W	Build a closed circular Civil 3D alignment (the circulating-carriageway centreline) from an exact 4-quadrant circle, and - if lane_width > 0 - inner and outer kerb offset alignments at +/- lane_width/2
civil3d_create_turning_path_arc	W	Create a single-arc Civil 3D alignment from a centre, design radius and start/end bearings (degrees, CCW)
civil3d_swept_path_envelope	W	Draw the inner + outer boundary polylines of a swept path by walking an alignment at two lateral offsets between two chainages

7.11 Parcels and sites (1)

Tool	R/W	Purpose
civil3d_list_sites_parcels	W	Sites with parcel number, area (m2 and ha) and perimeter; optional CSV for land schedules

7.12 Drafting, layers, styles and drawing utilities (18)

Tool	R/W	Purpose
civil3d_create_3d_polyline	W	Create a 3D polyline from [x, y, z] points (e.g. for breaklines)
civil3d_create_layer	W	Create a layer with a given colour index
civil3d_create_polyline	W	Create a 2D polyline (optionally closed) from [x, y] points
civil3d_create_text	W	Place single-line text or MText
civil3d_erase_entities	W	Erase objects by handle
civil3d_get_drawing_info	R	Drawing name, filename, version, object counts (alignments, corridors, etc.)
civil3d_get_drawing_settings	R	Drawing units, coordinate system code (e.g
civil3d_get_entity_info	R	Type, layer, name, extents, length and area for any object handle
civil3d_list_assemblies	R	List corridor assemblies available in the drawing
civil3d_list_layers	R	List all layers with state flags (frozen / off / locked / current)
civil3d_list_styles	R	Style names per collection (alignment, profile, surface, profile_view, section_view, point, corridor, label sets ...)

civil3d_measure_polylines	R	Total length and area of polylines grouped by layer, with wildcard layer filters
civil3d_run_command	W	ESCAPE HATCH: send an arbitrary AutoCAD/C3D command-line string to the document, exactly as if typed
civil3d_save_drawing	W	Save the active drawing
civil3d_set_current_layer	W	Make a layer the drawing's current/active layer
civil3d_set_object_style	W	Apply a named style to an alignment, profile, surface, corridor, or any styled object by handle
civil3d_zoom_extents	W	Zoom the active viewport to drawing extents
civil3d_zoom_to_alignment	W	Zoom the active viewport to the extents of an alignment, padded 10%

7.13 Design checks and reports (6)

Tool	R/W	Purpose
civil3d_alignment_overview_report	R	Summarise a horizontal alignment: overall geometry, breakdown of line / arc / spiral elements, station range and the profiles it carries
civil3d_check_alignment_design	R	Check every horizontal curve against R_min for the design speed and e_max (AASHTO 2018 metric), and flag short tangents between reverse curves and broken-back curves
civil3d_check_profile_design	R	Check vertical curves for minimum K (crest stopping sight distance, sag headlight) and minimum length, and tangents for max/min grade
civil3d_drawing_inventory_report	R	Whole-drawing dashboard: counts + names of alignments, profiles are omitted for brevity, surfaces, corridors, pipe networks and junctions present in the active Civil 3D drawing
civil3d_profile_design_report	R	Summarise a profile's vertical geometry: PVI table, grade breaks, curve count/lengths, min/max elevation and design stats
civil3d_quantity_report	R	Compute and format the cut / fill / net volume between two surfaces (base vs comparison) using the bridge volume surface, as markdown or json

7.14 Engineering calculators (no Civil 3D needed) (7)

Tool	R/W	Purpose
calc_average_end_area	R	Volumes and mass-haul ordinates from a station / cut area / fill area table
calc_cogo_forward	R	Coordinates from a point, whole-circle bearing and distance
calc_cogo_inverse	R	Grid bearing (whole circle and DMS) and distance between two points
calc_design_controls	R	Stopping sight distance, minimum crest/sag K, minimum radius and f_max for a design speed (AASHTO 2018 metric), with the formulas and assumptions used

`calc_horizontal_curve`	R	Tangent length, arc length, long chord, external, middle ordinate (and TC/CT stations)
`calc_superelevation_runoff`	R	Superelevation runoff and tangent runout lengths (AASHTO 2018 eq. 3-23/3-24)
`calc_traverse_bowditch`	R	Compute a traverse from bearings and distances, report misclosure and precision, and adjust by the Bowditch (compass) rule

---

\newpage

8. Argument conventions

8.1 Stations and offsets

- **Stations** are in drawing units (metres for the Ugandan / metric case).
- **Offsets** are **+ left, – right** of the direction of travel.
- **Skew** for the culvert tool is measured in **degrees from perpendicular**; +ve rotates the pipe CCW looking down.

8.2 Names vs handles

Keys like `alignment`, `surface`, `network` accept either:

- A **name** (case-insensitive but otherwise exact match), or
- An AutoCAD **handle** (`<a8f3>` style).

If the same name exists in multiple sites, use the handle.

8.3 File paths

CSV input/output paths must be **absolute Windows paths** (`C:\...`). The bridge runs inside Civil 3D's process - relative paths resolve against Civil 3D's working directory, which is rarely what you want.

8.4 Layer naming

The bridge follows AIA / NCS-ish conventions for any layers it auto-creates:

- `C-ROAD-*` - road geometry, marking
- `C-SIGN-*` - signage
- `C-DRAIN-*` - drainage / culverts
- `C-TOPO-*` - topographic data

If a target layer doesn't exist when a tool needs it, the bridge creates it with a sensible ACI

colour.

\newpage

9. Stubbed tools in 0.2.0

These tools exist in the registry and Claude can attempt to call them, but the underlying Civil 3D 2026 .NET API entry points were renamed or restructured enough that a faithful port is deferred. Calling them returns a clear `ToolException` with a "not yet ported" message.

Tool	What changed in C3D 2026
`create_culvert_at_chainage`	`Network.AddNetworkPart()` + `NetworkPartType` enum gone; replaced with typed `AddLinePipe` / `AddStructure`.
`create_alignment_from_polyline`	The 8-arg `Alignment.Create(...)` overload gone; the new signature requires a different style/site resolution.
`add_vertical_curve`	`ProfileEntityCollection.AddFixedParabolaByLength` gone; replaced with `AddFreeSymmetricParabolaByPVIAndCurveLength` and friends.
`export_corridor_sections`	`Baseline.CalculatedStationList` gone; replaced with `GetAppliedAssemblyAtStation` per-station calls.
`compute_quantity_takeoff`	`Section.SampleSectionPoints` removed; only the `SectionPoints.Location` Point3d is exposed.

All 59 other tools work end-to-end against the live drawing.

\newpage

10. Troubleshooting

"The hammer icon doesn't show civil3d"

Most likely cause: the JSON config is malformed. Validate it at jsonlint.com (paste in the file contents). The two most common errors:

- Backslashes not doubled in the `command` path.
- Two top-level objects glued together - Claude shows *"Could not load app settings - Unexpected token"*.

Then fully quit Claude Desktop and reopen.

"Bridge call failed: Connection refused"

The C# add-in isn't loaded in Civil 3D. Check:

1. Civil 3D is actually running.
2. A DWG is open.
3. `MCPSTATUS` at the C3D command line confirms the bridge is listening.
4. The bundle deployed cleanly to `%APPDATA%\Autodesk\ApplicationPlugins\Civil3DMcpBridge.bundle\Contents\`.

If the bundle is there but the bridge isn't loading, try `NETLOAD` on the DLL manually - the load failure will surface in Civil 3D's command line.

"ToolException: not yet ported to C3D 2026 API"

You hit one of the 5 stubbed tools - see section 9. Use the underlying Civil 3D UI for now or open an issue in the repo.

"active drawing is not a Civil 3D document"

You opened a plain AutoCAD DWG instead of a Civil 3D one (different template). Open or create a drawing from a Civil 3D template (the file opens with a "Civil 3D" ribbon).

Logs

- **Claude Desktop MCP log**: `%APPDATA%\Claude\logs\mcp.log`
- **Civil 3D command line history**: `Ctrl-F2` opens the AutoCAD text window; the bridge writes its load message at startup.
- **Bridge HTTP log**: stdout of the Civil 3D process - visible if you launched Civil 3D from a terminal.

Reset

To uninstall:

```
# 1. Remove the bundle
Remove-Item -Recurse "$env:APPDATA\Autodesk\ApplicationPlugins\Civil3DMcpBridge.bundle"

# 2. Remove the Python package
pip uninstall civil3d-mcp

# 3. Edit %APPDATA%\Claude\claude_desktop_config.json and delete
#    the "civil3d" entry under mcpServers.

# 4. Restart Civil 3D and Claude Desktop.
```

\newpage

Appendix A: File locations

Path	Purpose
`%APPDATA%\Autodesk\ApplicationPlugins\Civil3DMcpBridge.bundle\`	Deployed C# add-in bundle
`%APPDATA%\Claude\claude_desktop_config.json`	Claude Desktop MCP server registry
`%APPDATA%\Claude\logs\mcp.log`	Claude Desktop MCP runtime log
`<Python>\Scripts\civil3d-mcp.exe`	Python MCP server entry point
`C:\Program Files\Autodesk\AutoCAD 2026\C3D\AeccDbMgd.dll`	Civil 3D 2026 managed API

Appendix B: Versioning

This manual covers **civil3d-mcp 0.4.0**. The version is reported by both `civil3d_bridge_health` and the deployed DLL's metadata.

Appendix C: License & support

Internal / personal-consultancy use. No warranty. The bridge runs only on `127.0.0.1` and does not listen on any external interface.

For deeper architecture and "how to add a tool" notes, see `docs/ARCHITECTURE.md` in the repo.